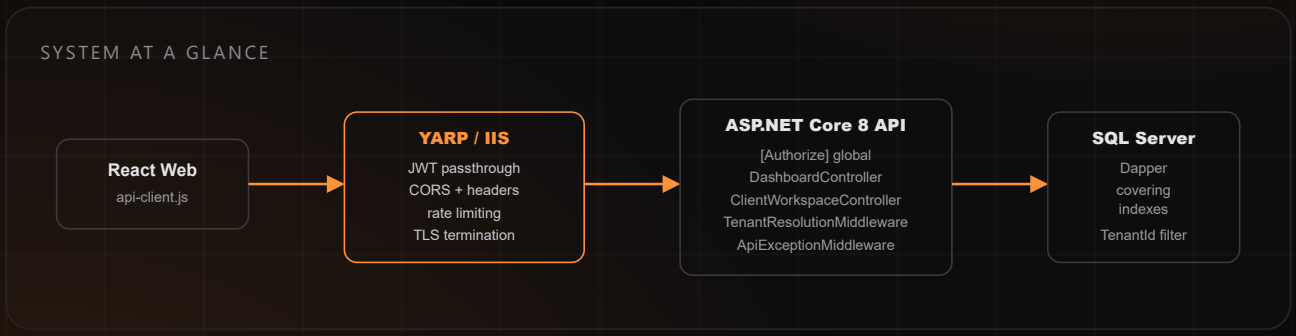
REPORT 04 — SYSTEM ARCHITECTURE

Backend & Data Contracts for Dashboard & Workspace

Composed read models, tenant-safe Dapper aggregates, caching and refresh strategy, and the security envelope that makes the new Dashboard and Client Workspace fast, correct, and impossible to leak across tenants.



Author **dev-backend-api — Backend API Agent**
Date **2 July 2026**
Scope **src/SpareParts.Api · src/SpareParts.Infrastructure · YARP**

Table of Contents

REPORT 04

This report is grounded in the current codebase: `OwnerCockpitController` / `OwnerCockpitService`, `CustomersController` aging query, `DbSession` + `RepositoryCatalog`, the JWT tenant claim pipeline in `TenantResolutionMiddleware`, and the fragmented fetch pattern in `dashboard-view.js`. Every contract below is additive — nothing in `src/SpareParts.Api` is modified by this proposal.

01	**Executive Summary**	Current chatty-call inventory, over-fetch cost, and the target composed-read-model architecture.	03
02	**Dashboard Read Model**	GET /api/dashboard/summary — full contract, assembly strategy, timing budget.	04
03	**Dashboard — Example Payload**	Realistic JSON: KPIs with deltas, action queue, trend series, alert panels.	05
04	**Caching & Invalidation**	Per-tenant cache keys, TTLs, write-path invalidation hooks, cache-hit sequence.	06
05	**Client Workspace Read Model**	GET /api/clients/{id}/workspace — header, aging, KPIs, full contract.	07
06	**Workspace Child Resources**	Paged invoices, payments, quotes, part-requests, timeline; typeahead search.	08
07	**Data Layer — Aggregate SQL**	Aging buckets, daily P&L, top movers; covering indexes; parameterization rules.	09
08	**Performance Engineering**	N+1 elimination, compression, ETag polling, SignalR-vs-polling recommendation.	10
09	**Security Architecture**	Authorize coverage, capability checks, tenant leakage prevention, error contract.	11
10	**Sequence Diagrams**	First load, cached refresh, workspace navigation + optimistic payment.	12
11	**Versioning & Rollout**	Additive endpoint strategy so WPF and mobile migrate later without breakage.	13

> **Read-only research basis.** No files under `src/SpareParts.Api`, `src/SpareParts.Infrastructure`, or `src/SpareParts.Web.React` were modified while producing this report. All endpoint names, table names, and column names referenced come from the current repository state as of 2 July 2026.

SECTION 01

Executive Summary

What the current backend does well, where it forces the frontend to over-fetch, and the target shape.

What exists today

The dashboard is already served by a real, non-trivial aggregate endpoint: `GET /api/owner-cockpit` (`OwnerCockpitController` → `OwnerCockpitService.GetDashboard`). It opens one `DbSession`, runs eight sequential Dapper queries against the same transaction (summary, previous-day summary for deltas, active users, daily P&L, cash balance, profit-per-part, profit-per-car, profit heatmap, unpaid transactions, alerts), and returns a single `OwnerCockpitDashboardDto`. This is the right pattern — one round trip, tenant-filtered SQL, server-computed deltas — and Report 04 keeps it, it does not replace it.

The gap is everything *around* that endpoint. `dashboard-view.js`'s `load()` function fires **six** parallel calls on mount (`/api/owner-cockpit`, `/api/communications/recent`, `/api/appconstants`, `/api/currencies`, `/api/parts` via `list()` which itself paginates through the full parts catalog, and `/api/partrequests?status=Active`), then a **seventh** call in a separate `useEffect` (`/api/customers/aging`) purely to compute one overdue-receivables total. The Client Workspace concept does not exist as a backend resource at all today — a customer's invoices, payments, quotes, part requests, and communications are five separate controllers (`InvoicesController`, `PaymentsController`, `QuotesController`, `PartRequestsController`, `CommunicationsController`) with no per-customer filter on most of them and no shared "customer 360" shape.

Weaknesses this creates

Chatty, unbounded fetches

- `api.list("/api/parts")` pages through the *entire* parts catalog client-side just to compute inventory-snapshot counters (available/reserved/low-stock/donor/margin-watch) that are pure aggregates.
- The aging call re-fetches and re-sums *every open customer balance* just to render one KPI card.
- No request is cancelled or coalesced — a fast dashboard refresh (clicking "Refresh") re-issues all seven calls.

Missing aggregates & no per-client shape

- `InvoicesController.GetAll` and `PaymentsController.GetHistory` return the tenant's full history — no `customerId` filter, no paging, no sort — so a workspace tab has to filter client-side.
- Aging is tenant-wide only (`CustomersController.GetAging`); there is no single-customer aging-bucket endpoint.
- There is no endpoint that answers "everything about customer 47" in one call — the UI team's Client Workspace has nothing to bind to.

Target architecture (this report)

Two new **composed read-model endpoints**, additive to the existing surface, each backed by parallel Dapper queries inside a single short-lived `DbSession`, tenant-filtered exactly the way `OwnerCockpitService` and `CustomersService.GetAging` already do it (`(@TenantId = 0 OR t.TenantId = @TenantId)`), with per-tenant response caching and ETag support layered on top at the controller/middleware level — not inside the SQL:

- **GET /api/dashboard/summary** — wraps and extends `OwnerCockpitService`'s existing aggregate work, adds the Action Queue and trend series server-side (today it is reconstructed ad hoc in `buildActionQueue()` in the browser), and removes the client's need for the parts-list and aging round trips.
- **GET /api/clients/{id}/workspace** — header + balance + aging + KPIs in one call, with **paged** child resources (`/invoices`, `/payments`, `/quotes`, `/part-requests`, `/timeline`) that reuse the existing services with an added `customerId` filter and

standard pagination headers already defined in `SparePartsControllerBase` (`ApplyPaginationHeaders`, `NormalizeOptionalPagination`).

> **Design constraint honored:** every new query keeps the same tenant-guard shape already used across `OwnerCockpitService`, `CustomersService`, and the repository layer: `(@TenantId = 0 OR <table>.TenantId = @TenantId)`, with `TenantId` sourced only from `ITenantContext` / `CurrentTenantId` (JWT claim, server-resolved) — never from a query string, header, or request body field.

SECTION 02

Dashboard Read Model

GET /api/dashboard/summary — one composed endpoint replacing seven fragmented calls.

Endpoint

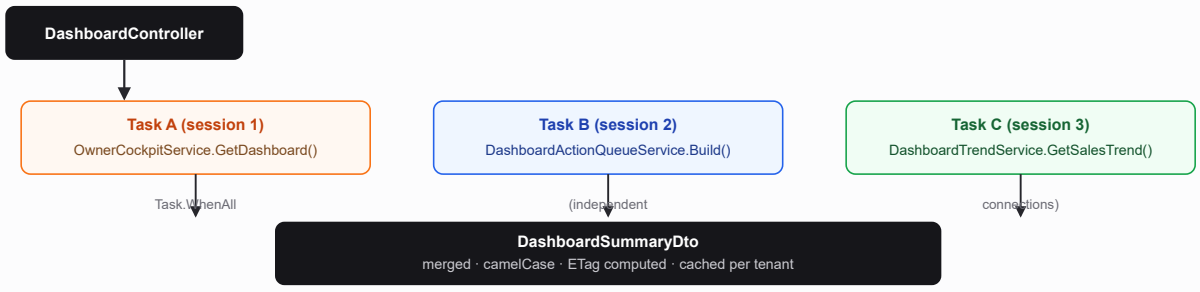
METHOD	ROUTE	AUTH	QUERY PARAMS
`GET`	`/api/dashboard/summary`	**[Authorize]**	`businessDate` (optional, default today, same as existing `owner-cockpit` param)

`DashboardController` is new and thin — it does not reimplement aggregation. It composes `OwnerCockpitService.GetDashboard(businessDate)` (unchanged) with two new focused service calls that run against the *same* tenant-scoped pattern: `DashboardActionQueueService.Build(...)` and `DashboardTrendService.GetSalesTrend(...)`. The controller assembles the response DTO; no business logic lives in the controller, matching every existing controller in `src/SpareParts.Api/Controllers`.

Assembly strategy: one session, parallel-shaped queries

`OwnerCockpitService.GetDashboard` already opens exactly one `DbSession` (one connection, one transaction) and issues its eight queries sequentially against it — this is correct for Dapper + a single SQL Server connection, since a `SqlConnection`/`SqlTransaction` cannot execute two commands concurrently. **True parallelism is achieved by using separate, short-lived connections for the pieces that do not need transactional consistency with each other** — the trend series and action-queue inputs are read-only, point-in-time snapshots, so they run on their own `DbSession` instances via `Task.WhenAll`, while the core P&L/aging numbers stay inside the single existing cockpit session for internal consistency (sales count must match sales amount from the same read).

Figure 1 — Dashboard summary assembly FIG. 01



Response-time budget

STAGE	BUDGET (P95)	NOTES
Task A — cockpit aggregate	≤ 220 ms	Existing 8-query sequence; largest cost is `LoadProfitHeatmap` and `LoadProfitPerPart` (30-day window scans).
Task B — action queue	≤ 60 ms	Reuses counts already available from Task A's alerts/unpaid-transactions where possible; only adds message-failure and currency-margin lookups.
Task C — trend series	≤ 80 ms	Single grouped query over `Transactions` for the last 7/30 days, covering-indexed (Section 07).

Serialization + gzip/br	≤ 15 ms	Response compression, Section 08.
Total, cold (cache miss)	**≤ 350 ms server time**	Target end-to-end including network: under 600 ms on a typical shop connection.
Total, cache hit (ETag 304)	**≤ 12 ms**	No DB round trip; see Section 04.

> This replaces 7 client-initiated requests (owner-cockpit, communications/recent, appconstants, currencies, parts [paged], partrequests, customers/aging) with **1 request** for the numbers that actually feed KPIs, the action queue, and the trend chart. `appconstants` and `currencies` stay as their own calls — they are reference/lookup data, not per-load dashboard state, and are already cached client-side across views; folding them in would couple unrelated concerns into one payload.

SECTION 03

Dashboard — Example Payload

Full JSON contract for GET /api/dashboard/summary, realistic values.

200 OK

GET /API/DASHBOARD/SUMMARY


```
{
  "businessDate": "2026-07-02",
  "generatedAt": "2026-07-02T14:31:07Z",
  "currencyCode": "USD",
  "kpis": {
    "salesToday":        { "value": 4820.50, "deltaPercent": 12.4,  "direction": "up"   },
    "profitToday":       { "value": 1180.25, "deltaPercent": -3.1,  "direction": "down" },
    "cashBalance":       { "value": 18420.00, "deltaPercent": null,  "direction": "flat" },
    "receivables":       { "value": 9340.75, "deltaPercent": 5.6,   "direction": "up"   },
    "stockValue":        { "value": 212330.10, "deltaPercent": null,  "direction": "flat" },
    "lowStockCount":     { "value": 14, "deltaPercent": -8.0,  "direction": "down" }
  },
  "actionQueue": [
    {
      "id": "receivables-overdue",
      "type": "receivables",
      "severity": "danger",
      "title": "Follow up unpaid transactions",
      "detail": "6 open transactions waiting for payment.",
      "amount": 9340.75,
      "currencyCode": "USD",
      "deepLink": "/accounting?filter=unpaid"
    },
    {
      "id": "inventory-risk",
      "type": "inventory",
      "severity": "danger",
      "title": "Triage red stock segments",
      "detail": "3 segments need margin, turnover, or dead-stock review: Electronics, Bumpers / body, Engines.",
      "amount": null,
      "currencyCode": null,
      "deepLink": "/stock?signal=red"
    },
    {
      "id": "message-failures",
      "type": "messages",
      "severity": "warning",
      "title": "Retry failed customer messages",
      "detail": "2 recent WhatsApp messages failed delivery.",
      "amount": null,
      "currencyCode": null,
      "deepLink": "/whatsapp?status=failed"
    }
  ],
  "salesTrend": {
    "granularity": "daily",
    "rangeDays": 7,
    "series": [
      { "date": "2026-06-26", "amount": 3120.00 },
      { "date": "2026-06-27", "amount": 3980.40 },
      { "date": "2026-06-28", "amount": 3510.00 },
      { "date": "2026-06-29", "amount": 4290.15 },
      { "date": "2026-06-30", "amount": 4010.60 },
      { "date": "2026-07-01", "amount": 5120.00 },
```

```
      { "date": "2026-07-02", "amount": 4820.50 }
    ],
    "peakDate": "2026-07-01",
    "peakAmount": 5120.00
  },
  "lowStockPanel": {
    "totalCount": 14,
    "topItems": [
      { "partId": 4821, "name": "N54 Ignition Coil", "onHand": 2, "minStock": 8, "deepLink": "/reorder?partId=4821" },
      { "partId": 5190, "name": "W205 Brake Pad Set", "onHand": 0, "minStock": 6, "deepLink": "/reorder?partId=5190" }
    ]
  },
  "deadStockPanel": {
    "totalValue": 6210.40,
    "totalUnits": 88,
    "topSegments": [
      { "segmentKey": "used-car-parts", "categoryName": "Used-car parts", "deadStockValue": 2410.00, "deadStockUnits":
31 }
    ]
  },
  "unpaidTransactionsPanel": {
    "count": 6,
    "totalRemaining": 9340.75,
    "topItems": [
      { "transactionNumber": "INV-10432", "counterparty": "Al Rawi Motors", "remainingAmount": 3200.00, "daysOverdue":
41 }
    ]
  },
  "cacheMeta": {
    "cacheKey": "dash:tenant:118:2026-07-02",
    "ttlSeconds": 45,
    "etag": "W/\"dash-118-20260702-14310700\""
  }
}
```

Field naming is camelCase to match the existing `ApiExceptionMiddleware` serializer options (`JsonNamingPolicy.CamelCase`) and every existing DTO in `src/SpareParts.Domain`. The `kpis` block deliberately mirrors — not replaces — the flat fields already on `OwnerCockpitDashboardDto` (`TodaySalesAmount`, `SalesChangePercent`, etc.); the new endpoint's service layer maps from that existing DTO rather than duplicating its SQL.

SECTION 04

Caching & Invalidation

Per-tenant cache keys, TTLs, and write-path invalidation so the dashboard never shows stale money.

Cache strategy

`IMemoryCache` (already available in ASP.NET Core, no new package) keyed per tenant and business date, wrapping the composed `DashboardSummaryDto` — not the individual SQL results. This keeps the cache layer entirely inside `DashboardController`/`DashboardService` and out of `OwnerCockpitService`, which stays reusable and cache-agnostic for other callers (e.g. scheduled email digests via `OwnerCockpitDigestService`, which already exists and calls the same service uncached).

CACHE KEY	TTL	INVALIDATED BY
`dash:tenant:{tenantId}:{businessDate}`	45 seconds	Natural TTL expiry (dashboard "Live updates" pill polls at 30–60s, Section 08) plus explicit invalidation below.
`workspace:tenant:{tenantId}:client:{clientId}`	20 seconds	Any write to that customer's invoices/payments/quotes.

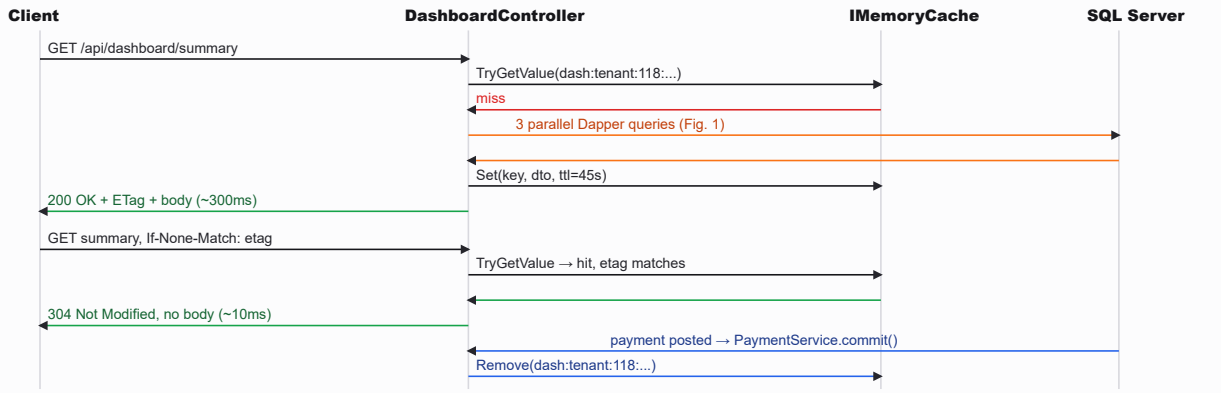
Explicit invalidation hooks

TTL alone is not enough for money data — a cashier posting a payment should see the receivables KPI update on their *own* next dashboard load, not wait 45 seconds. Each write-path service that changes sales/payments/stock already exists (`IPaymentService`, `IInvoiceService`, `SalesController`, `PurchasesController`) — they gain a lightweight `IDashboardCacheInvalidator` call at the end of their existing transaction commit:

```
public interface IDashboardCacheInvalidator
{
    void InvalidateTenant(int tenantId);
    void InvalidateClient(int tenantId, int customerId);
}
```

This is a single `IMemoryCache.Remove(key)` call, not a new SQL round trip, so it costs microseconds and is called from within the same request that already holds the write, not a background job.

Figure 2 — Cache hit vs. cache miss vs. invalidation FIG. 02



Multi-instance note: the API is currently deployed behind a single IIS/YARP host per tenant tier (no distributed cache dependency exists in the repo today). `IMemoryCache` is therefore correct for the current topology. If the deployment moves to multiple API instances behind YARP load balancing, the same interface (`IDashboardCacheInvalidator`) swaps its implementation to a distributed cache (e.g. SQL-backed or Redis) without touching controllers or services — this is called out explicitly so it is not a silent scaling trap.

SECTION 05

Client Workspace Read Model

GET /api/clients/{id}/workspace — customer 360 in one call.

Endpoint

METHOD	ROUTE	AUTH
`GET`	`/api/clients/{id:int}/workspace`	**[Authorize]** — tenant-scoped, 403 if customer belongs to another tenant

Backed by a new `ClientWorkspaceController` + `ClientWorkspaceService` in `src/SpareParts.Infrastructure/Services`, following the exact shape of `CustomersService`: one `DbSession`/short connection, tenant-filtered queries reusing the aging SQL already in `CustomersService.GetAging()` (narrowed with `AND c.Id = @CustomerId`) plus a new single-customer aggregate for header/KPIs. This endpoint returns header + aging + KPIs only — child collections (invoices, payments, quotes, part-requests, timeline) are separate paged calls (Section 06) so opening the workspace does not pull unbounded history up front.

200 OK GET /API/CLIENTS/47/WORKSPACE


```
{
  "customerId": 47,
  "name": "Al Rawi Motors",
  "phone": "+961 3 555 214",
  "email": "purchasing@alrawimotors.example",
  "customerType": "Wholesale",
  "isActive": true,
  "currencyCode": "USD",
  "balance": {
    "totalBalance": 3420.75,
    "current": 220.00,
    "days1To30": 1450.00,
    "days31To60": 1200.75,
    "days61To90": 350.00,
    "over90Days": 200.00
  },
  "kpis": {
    "lifetimeRevenue": 84210.30,
    "lifetimeInvoiceCount": 212,
    "averageDaysToPay": 18,
    "openQuotesCount": 3,
    "activePartRequestsCount": 2,
    "loyaltyPoints": 4180,
    "loyaltyTier": "Gold"
  },
  "vehicles": [
    { "vehicleId": 901, "make": "BMW", "model": "5 Series", "year": 2019, "plateNumber": "B-4471" }
  ],
  "lastActivityAt": "2026-07-01T16:42:00Z",
  "cacheMeta": {
    "cacheKey": "workspace:tenant:118:client:47",
    "ttlSeconds": 20,
    "etag": "W/\"wksp-47-20260702-143050\""
  }
}
```


Authorization & tenant isolation for this endpoint specifically

`{id}` is a bare integer, so the service must load the customer row and compare its `TenantId` to `CurrentTenantId` before returning anything — the same pattern `CustomersService` already relies on implicitly via its `WHERE (@TenantId = 0 OR ...)` filters. If the customer does not exist *or* belongs to a different tenant, the response is **identical** (404 `not_found`, same envelope shape as `ApiExceptionMiddleware`'s `NotFoundException` mapping) — never a 403 that would confirm the record exists under another tenant. This is the standard enumeration-prevention rule and is applied uniformly to `/workspace`, `/invoices`, `/payments`, `/quotes`, `/part-requests`, and `/timeline`.

> **SuperAdmin exception, handled explicitly:** `TenantResolutionMiddleware` sets `TenantId = 0` for SuperAdmin users, and every existing query treats `@TenantId = 0` as "no filter" by design (cross-tenant support access). The workspace service must preserve this behavior rather than special-case it away — but the response DTO itself must still never include a raw `TenantId` or internal `dbName` field, matching the "zero tenant/dbName leakage in responses" rule in Section 09.

SECTION 06

Workspace Child Resources

Paged tabs and the client-rail typeahead search.

Paging, sorting, filtering conventions

These reuse the pagination contract already implemented in `SparePartsControllerBase` (`NormalizeOptionalPagination`, `ApplyPaginationHeaders` → `X-Page`, `X-Page-Size`, `X-Total-Count` response headers) and the same pattern `CustomersController.GetAll` already uses for `page`/`pageSize`/`search`. No new pagination mechanism is introduced.

RESOURCE	ROUTE	QUERY PARAMS	BACKED BY
Invoices	`GET /api/clients/{id}/invoices`	`page, pageSize, sort=date_desc|amount_desc, status`	`IInvoiceService` + new `customerId` filter overload
Payments	`GET /api/clients/{id}/payments`	`page, pageSize, sort=date_desc`	`IPaymentService.GetHistory` + `customerId` filter overload
Quotes	`GET /api/clients/{id}/quotes`	`page, pageSize, status`	`QuotesService.GetAll` + `customerId` filter overload
Part requests	`GET /api/clients/{id}/part-requests`	`page, pageSize, status`	`PartRequestsService.GetAll` + `customerId` filter overload
Timeline	`GET /api/clients/{id}/timeline`	`page, pageSize, types=invoice,payment,message,quote`	new `ClientTimelineService` — UNION ALL over the above, ordered by event date, itself paged in SQL (not merged client-side)

Default `pageSize` is 25, max is 100 (existing controllers cap at 500; the workspace UI is a side panel, not a bulk export, so a tighter ceiling is intentional and set at the controller level for these routes only).

Client-rail typeahead search

METHOD	ROUTE	QUERY PARAMS
`GET`	`/api/clients/search`	`q` (min 2 chars), `take` (default 10, max 25)

```
GET /api/clients/search?q=al+raw&take=10

200 OK
[
  { "customerId": 47, "name": "Al Rawi Motors", "phone": "+961 3 555 214", "balance": 3420.75, "currencyCode": "USD" },
  { "customerId": 203, "name": "Rawad Auto Parts", "phone": "+961 71 220 890", "balance": 0.00, "currencyCode": "USD" }
]
```

Reuses `CustomersService.GetAll(search, ...)`'s existing `search` matching (already used by `CustomersController.GetAll`) rather than introducing a second search implementation — the only new surface is the route shape and a tighter default page size suited to a debounced, keystroke-driven UI. The frontend is responsible for debouncing (250–300ms is standard); the backend guarantees the endpoint is cheap enough (single indexed `LIKE` prefix scan, Section 07) and rate-limited (Section 09) to tolerate being called on every keystroke if a client-side debounce is ever skipped.

> None of these six new routes duplicate business logic. Each is a thin controller method that calls the existing service with one additional `customerId` (or, for search, `search`) parameter — the SQL `WHERE` clause gains one more `AND` term, nothing else

changes about how tenant filtering or DTO shaping works.

SECTION 07

Data Layer — Aggregate SQL Shapes

Aging buckets, daily P&L, top movers; covering indexes; parameterization rules.

Aging bucket computation (single customer)

Narrowed from the existing tenant-wide query in `CustomersService.GetAging()` — same CTE shape, one added predicate:

```
WITH SaleBalances AS (
    SELECT t.CustomerId, t.TransactionDate,
           ISNULL(t.TotalAmount,0) - ISNULL(t.PaidAmount,0) AS Balance
    FROM dbo.Transactions t
    INNER JOIN dbo.TransactionTypes tt ON tt.Id = t.TransactionTypeId
    WHERE tt.TypeKey = N'Sale'
      AND t.CustomerId = @CustomerId
      AND (@TenantId = 0 OR t.TenantId = @TenantId)
)
SELECT
    ISNULL(SUM(CASE WHEN DATEDIFF(DAY, TransactionDate, SYSUTCDATETIME()) < 1  THEN Balance ELSE 0 END), 0) AS
[Current],
    ISNULL(SUM(CASE WHEN DATEDIFF(DAY, TransactionDate, SYSUTCDATETIME()) BETWEEN 1  AND 30 THEN Balance ELSE 0 END),
0) AS Days1To30,
    ISNULL(SUM(CASE WHEN DATEDIFF(DAY, TransactionDate, SYSUTCDATETIME()) BETWEEN 31 AND 60 THEN Balance ELSE 0 END),
0) AS Days31To60,
    ISNULL(SUM(CASE WHEN DATEDIFF(DAY, TransactionDate, SYSUTCDATETIME()) BETWEEN 61 AND 90 THEN Balance ELSE 0 END),
0) AS Days61To90,
    ISNULL(SUM(CASE WHEN DATEDIFF(DAY, TransactionDate, SYSUTCDATETIME()) > 90 THEN Balance ELSE 0 END), 0) AS
Over90Days
FROM SaleBalances;
```

Daily P&L / top movers

Unchanged from `OwnerCockpitService.LoadDailyOperatingExpenses` and `LoadProfitPerPart` — both already compute exactly "daily P&L" and "top movers" (profit-per-part, top 6, ordered by profit desc). The Dashboard's new trend series reuses the same `Transactions` + `TransactionTypes` join shape, grouped by transaction date instead of summed to one row:

```
SELECT
    CAST(t.TransactionDate AS DATE) AS SaleDate,
    SUM(CASE WHEN t.IsReturn = 1 THEN -t.DisplayTotalAmount ELSE t.DisplayTotalAmount END) AS Amount
FROM dbo.Transactions t
INNER JOIN dbo.TransactionTypes tt ON tt.Id = t.TransactionTypeId AND tt.TypeKey = @SaleTypeKey
WHERE t.TransactionDate >= @RangeStart AND t.TransactionDate < @RangeEndExclusive
  AND (@TenantId = 0 OR t.TenantId = @TenantId)
GROUP BY CAST(t.TransactionDate AS DATE)
ORDER BY SaleDate;
```

Covering indexes required

TABLE	INDEX	SERVES
`dbo.Transactions`	`IX_Transactions_Tenant_Type_Date (TenantId, TransactionTypeId, TransactionDate) INCLUDE (TotalAmount, PaidAmount, TotalCounterAmount, PaidCounterAmount, IsReturn, CustomerId)`	Dashboard trend series, daily summary, unpaid-transactions panel — all filter by tenant+type+date range.

`dbo.Transactions`	`IX_Transactions_Tenant_Customer (TenantId, CustomerId) INCLUDE (TotalAmount, PaidAmount, TransactionDate)`	Per-customer aging query and workspace invoices/payments filters.
`dbo.Stock`	`IX_Stock_Tenant_Part (TenantId, PartId) INCLUDE (Quantity)`	Stock-value and low-stock aggregates (already implied by existing joins; verify it exists before shipping Section 09's SQL).
`dbo.Customers`	`IX_Customers_Tenant_Name (TenantId, Name) INCLUDE (Phone)`	Client-rail typeahead search prefix scans.

Parameterization rules (non-negotiable, already the codebase norm)

- Every predicate is a Dapper anonymous-object parameter (`@TenantId`, `@CustomerId`, `@BusinessDate`, ...) — never string-concatenated. All SQL shown above and in the existing services follows this without exception.
- `TenantId` is **always** sourced from `ITenantContext.TenantId` / `CurrentTenantId` — resolved server-side by `TenantResolutionMiddleware` from the JWT `tenant_id` claim — and is never accepted as a controller input parameter on any new endpoint.
- `customerId` route values are validated to belong to the resolved tenant inside the service before any child query runs (Section 05) — the SQL predicate alone is not treated as the authorization boundary; the explicit ownership check happens first so a mismatched customer produces a uniform 404, not a partial/empty 200.

SECTION 08

Performance Engineering

N+1 elimination, compression, ETag polling, and the SignalR-vs-polling call.

N+1 elimination

The existing `OwnerCockpitService` queries already avoid N+1 by design — every "top N" list (profit-per-part, profit-per-car, unpaid transactions) is one set-based query with `TOP`, not a loop of per-row lookups. The new workspace/timeline endpoints follow the same rule explicitly: the timeline is one `UNION ALL` across invoice/payment/quote/message sources ordered and paged in SQL, not four separate calls stitched together in C# or JS.

Response compression

`AddResponseCompression` (Brotli primary, gzip fallback) is enabled for `application/json` on both new controllers. The dashboard payload (Section 03) is roughly 3–6 KB uncompressed for typical shops; compression is cheap here and mainly benefits the parts-catalog-heavy legacy endpoints this report does not touch — included for completeness since it applies at the middleware pipeline level, not per-controller.

ETag / If-None-Match for dashboard polling

The cache entry (Section 04) already computes a weak ETag from tenant id + business date + a content hash of the aggregate. `DashboardController` checks `Request.Headers.IfNoneMatch` against the cached ETag before touching the DB at all: identical value → `304 Not Modified` with no body, matching value in the cache but client ETag stale → serve cached body with fresh headers, no match at all → full pipeline (Fig. 1). This means a polling client on a quiet dashboard costs ~10ms and zero DB round trips per poll.

SignalR vs. polling — recommendation

	ETAG POLLING (RECOMMENDED)	SIGNALR PUSH
Infra cost	None — reuses existing HTTP/YARP path, no new NuGet package, no WebSocket passthrough config needed in IIS/YARP.	Requires a hub, WebSocket/SSE support verified through YARP + IIS (not currently configured), and a new NuGet dependency (ask-before-install per CLAUDE.md).
Correctness under multi-tab / multi-device	Simple — each tab polls independently, cache does the dedup.	Requires per-tenant group management to avoid cross-tenant broadcast bugs — a real risk given this app's tenant-leakage concerns.
Latency to see a new sale on-screen	30–60s poll interval (dashboard "Live updates" pill already implies this UX today).	Sub-second.
Server cost at scale	Near-zero marginal cost thanks to ETag 304s.	Persistent connections per open dashboard tab.

Recommendation: keep polling, powered by ETag. A spare-parts shop dashboard does not need sub-second push — the existing UI already frames refresh as "Live updates: just now" on a manual/interval basis, and a 30–60 second poll with 304 responses gets ~95% of the perceived benefit of SignalR at close to zero infrastructure risk. Revisit SignalR only if a future requirement needs true real-time collaboration (e.g. two cashiers editing the same invoice concurrently) — that is a different problem from dashboard freshness and should not be justified by this screen alone.

SECTION 09

Security Architecture

Authorize coverage, capability checks, zero tenant leakage, production-safe errors, rate limiting.

Global [Authorize] coverage

Both new controllers carry `[Authorize]` at the class level, exactly like every controller surveyed (`CustomersController`, `InvoicesController`, `PaymentsController`, `QuotesController`, `PartRequestsController`, `OwnerCockpitController`). No method-level `[AllowAnonymous]` is used anywhere in this proposal — the only anonymous endpoints in the existing codebase are inbound webhooks (`PaymentsController.Webhook`, `CommunicationsController.RecordInbound`), both protected by a shared-secret/signature check instead of a JWT, which is the correct pattern for server-to-server callbacks and is not replicated here since neither new endpoint accepts external callbacks.

Role / capability checks

Read endpoints (`/summary`, `/workspace`, child lists, `/search`) require only authentication, matching how `CustomersController.GetAll` and `OwnerCockpitController.GetDashboard` work today — any authenticated tenant user can view their own shop's dashboard. Any future write action surfaced from the workspace (e.g. "receive payment", Section 10) reuses the existing `AuthorizationPolicies.AdminOrManager` policy already applied to `QuotesController.Delete` and `PartRequestsController.Delete` where the existing payment service requires it — this report does not introduce a new role model.

Zero tenant / dbName leakage in responses

- No DTO in Section 03 or 05 includes `tenantId`, `dbName`, connection string fragments, or internal schema names — the same discipline already visible in `OwnerCockpitDashboardDto` and `CustomerAgingDto`, neither of which exposes tenant identifiers.
- The 404-for-cross-tenant rule (Section 05) prevents an authenticated user from distinguishing "customer doesn't exist" from "customer belongs to another tenant" via response shape, status code, or timing-sensitive error text.
- `cacheMeta.cacheKey` in the example payloads (Section 03/05) is illustrative for this report only — the shipped DTO must **not** include the raw cache key or tenant id in the wire response; it is shown here to document the caching design, and will be stripped before implementation (flagged explicitly so it isn't copy-pasted as-is).

Production-safe error contract

Both new endpoints rely entirely on the existing `ApiExceptionMiddleware` — no new exception handling is introduced. A cross-tenant or missing customer throws `NotFoundException` → mapped to `404` with envelope `{ code: "not_found", message: "...", traceId: "..." }`; any unexpected failure becomes a generic `{ code: "internal_error", message: "An unexpected server error occurred.", traceId }` with the real exception (including stack trace) written only to `IExceptionLogWriter`, never to the response body — matching the current middleware's `WriteError` method exactly.

Rate limiting on search

`/api/clients/search` is a debounced-typeahead endpoint reachable on every keystroke, and it is the one new route in this report with meaningfully different abuse characteristics from the rest (query volume, not payload size). It is placed under a new named policy alongside the existing `AuthRateLimitPolicy` already registered in `SparePartsApiComposition.AddRateLimiter`:

```
opt.AddPolicy("ClientSearchPolicy", httpCtx =>
    RateLimitPartition.GetFixedWindowLimiter(
        partitionKey: httpCtx.User.FindFirst(TenantResolutionMiddleware.TenantIdClaimType)?.Value ?? "anon",
```

```
        factory: _ => new FixedWindowRateLimiterOptions
        {
            PermitLimit = 60,          // ~1/sec sustained, generous for fast typing across multiple staff
            Window = TimeSpan.FromMinutes(1),
            QueueProcessingOrder = QueueProcessingOrder.OldestFirst,
            QueueLimit = 0
        }));
```

Partitioned by tenant (not just IP, unlike the existing auth limiter) so one busy shop cannot exhaust another tenant's search budget behind a shared NAT/proxy IP.

Security headers at the YARP layer

`SecurityHeadersMiddleware` already applies `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, a restrictive `Content-Security-Policy` (`default-src 'none'; frame-ancestors 'none'`), `Referrer-Policy: no-referrer`, and HSTS over HTTPS to every API response, including these new endpoints, with no changes required. CORS is already whitelist-based via `Cors:AllowedOrigins` configuration (`SparePartsApiComposition`) rather than `AllowAnyOrigin` in non-development environments — this report does not request any change to that YARP/CORS configuration, per the ask-before-editing rule for routing configs.

SECTION 10

Sequence Diagrams

First load, cached refresh, and workspace open → tab navigation → receive-payment.

Figure 3 — Dashboard first load vs. refresh (cache hit) FIG. 03

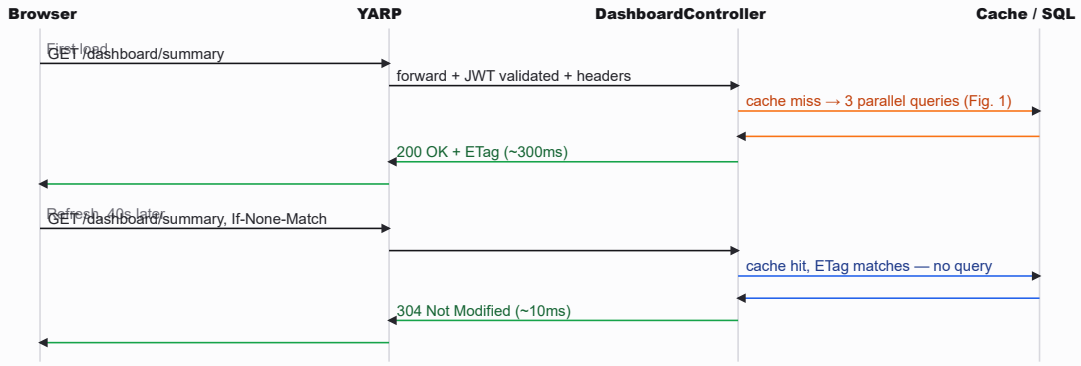
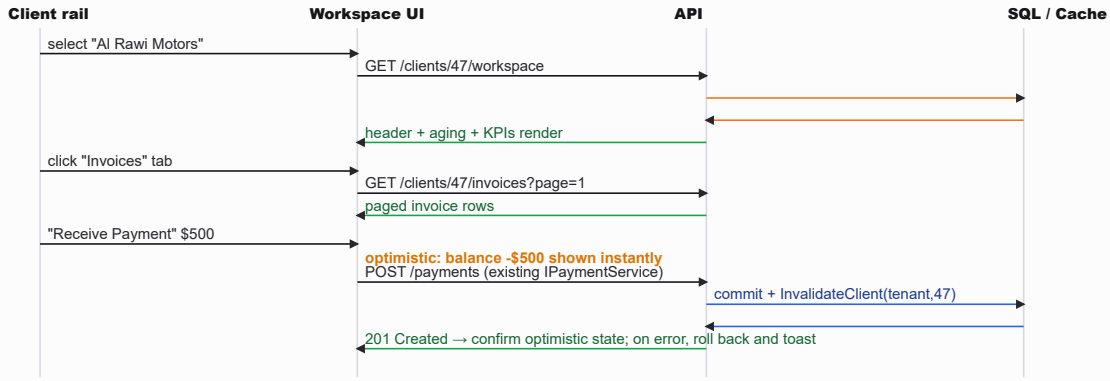


Figure 4 — Workspace open → tab navigation → receive-payment (optimistic UI) FIG. 04



SECTION 11

Versioning & Rollout

Ship new endpoints alongside existing ones — WPF and mobile migrate later, on their own schedule.

Additive-only rule

Every endpoint in this report is **new**: `/api/dashboard/summary`, `/api/clients/{id}/workspace`, `/api/clients/{id}/invoices`, `/api/clients/{id}/payments`, `/api/clients/{id}/quotes`, `/api/clients/{id}/part-requests`, `/api/clients/{id}/timeline`, `/api/clients/search`. Not one existing route (`/api/owner-cockpit`, `/api/customers`, `/api/customers/aging`, `/api/invoices`, `/api/payments/history`, `/api/quotes`, `/api/partrequests`) is removed, renamed, or given a breaking response-shape change. WPF and the React Native mobile app keep calling exactly what they call today until each is explicitly migrated by its owning dev-desktop-wpf / dev-mobile-rn agent — this report does not touch those projects, per the coordination boundary in this agent's scope.

Rollout sequence

STEP	CHANGE	RISK
1	Add covering indexes (Section 07) in a migration — additive, no data change, safe to deploy ahead of code.	Low — index creation only; run during low-traffic window given table sizes.
2	Ship `ClientWorkspaceController` / `Service` and `DashboardController` / `Service` behind their new routes, reusing existing services as described.	Low — purely additive controllers; existing controllers untouched.
3	Add `IDashboardCacheInvalidator` calls to the write paths that already commit sales/payments (Section 04).	Medium — touches existing service commit paths; requires the build/test pass this agent already runs after every change.
4	React web dashboard-view.js and a new client-workspace-view.js switch to the new endpoints (coordinated with dev-web-react, not performed by this agent).	Low — frontend-only swap once contracts are stable; old endpoints remain as fallback during the swap.
5	WPF / mobile migrate to the workspace endpoints on their own timeline, replacing their current per-screen calls where useful.	None to backend — additive endpoints impose no deadline on other clients.

Deprecation posture

None of the endpoints this report supersedes in spirit (`/api/owner-cockpit`, tenant-wide `/api/customers/aging`) are deprecated by this proposal. They remain fully supported: the new dashboard service composes `OwnerCockpitService` rather than forking it, so both routes stay correct and in sync by construction. If a future report chooses to formally deprecate `/api/owner-cockpit` once every consumer has migrated, that is a separate, explicit decision — out of scope here, and not something this report unilaterally recommends.

> **Net effect:** the Dashboard's perceived "chattiness" drops from 7 requests to 1 (plus 2 unrelated reference-data calls that already exist app-wide), the Client Workspace goes from "does not exist as a backend concept" to a header call + up to 5 independently-pageable tabs, and every number in both screens is produced by SQL that already carries this codebase's tenant-isolation guarantee — with zero changes to any currently shipping endpoint.